

Software Requirements Specification

for

CryptDragon System

Version 1.0

Prepared by:

Reem Ameen
Omnia Helmy
Rodaina Mohamed Elhady

Prepared for: Dr. Sherief Elshourbagy

Project Repository:

https://github.com/ReemAmeen/Crypt_Dragon

Contents

1	Introduction	3
1.1	Document Purpose	3
1.2	Product Scope	3
1.3	Intended Audience and Document Overview	3
1.4	Definitions, Acronyms and Abbreviations	4
2	Overall Description	5
2.1	Product Perspective	5
2.2	Product Functionality	5
2.3	Users and Characteristics	5
2.4	Operating Environment	5
2.5	Design and Implementation Constraints	6
2.6	Deployment	6
3	Specific Requirements	7
3.1	External Interface Requirements	7
3.1.1	User Interfaces	7
3.1.2	Hardware & Software Interfaces	7
3.2	Functional Requirements (Detailed Workflow)	7
3.3	Non-functional Requirements & Security Analysis	8
4	System Design	9
4.1	End-to-End Operation Sequence	9

4.2 Algorithm Comparison Summary	9
--	---

5 Scope of Extension	9
-----------------------------	----------

1 Introduction

Developing the CryptDragon project to implement a Hybrid Encryption Model aimed at securely transferring confidential files. Traditional encryption faces a dilemma: Asymmetric Encryption (RSA) is highly secure for sharing keys but is too slow and resource-heavy for large files, whereas Symmetric Encryption (AES) is incredibly fast and perfect for large files, but securely sharing the key over an untrusted network is dangerous.

CryptDragon solves this by combining the best of both worlds. It uses the lightning-fast AES to encrypt the actual file, and then uses the highly secure RSA to encrypt only the AES session key before sending it to the receiver.

There are basically 2 entities simulated in this system:

- Sender
- Receiver

1.1 Document Purpose

The document specifies the software requirement specification for the CryptDragon System. It serves to outline the architectural design, algorithmic choices, and the specific step-by-step workflow of the hybrid encryption model applied within a purely client-side web environment.

1.2 Product Scope

CryptDragon acts as a secure end-to-end (E2E) communication channel simulator between two entities. The project is built entirely on the client side, ensuring that no unencrypted data or private keys ever leave the user's browser. It demonstrates a modern, secure, and user-friendly approach to file encryption by leveraging the WebCrypto API, proving that complex cryptographic concepts can be abstracted into an elegant interface.

1.3 Intended Audience and Document Overview

The document is intended for:

- **Software Developers:** To understand the implementation of WebCrypto API and the hybrid logic.
- **Security Analysts:** To evaluate the Zero-Knowledge Transport architecture, OAEP padding, and client-side execution safety.

- **Project Evaluators & Instructors:** To assess the application of cryptographic theories into a functional web application.

1.4 Definitions, Acronyms and Abbreviations

Definitions & Acronyms:

- **RSA-OAEP:** Rivest-Shamir-Adleman with Optimal Asymmetric Encryption Padding (2048-bit). Used for encrypting the session key.
- **AES-CBC:** Advanced Encryption Standard in Cipher Block Chaining mode (128-bit). Used for encrypting file data.
- **IV (Initialization Vector):** A 16-byte random value used along with the AES key to ensure unique ciphertexts for identical files.
- **E2E:** End-to-End Encryption.
- **WebCrypto API:** Native HTML5 browser API used for secure, client-side cryptographic operations.

2 Overall Description

2.1 Product Perspective

The architecture simulates a secure end-to-end communication channel built entirely on the client side. The web application avoids backend processing of sensitive data; all cryptographic operations (generation, encryption, decryption) are performed within the browser's DOM memory context.

2.2 Product Functionality

The application provides a dual-dashboard interface (Sender and Receiver). Functionalities include:

- Generation of 2048-bit RSA Public/Private key pairs.
- Generation of random 128-bit AES session keys and 16-byte IVs.
- File processing via ArrayBuffers for high-speed AES encryption.
- Encryption of the AES Session Key using the RSA Public Key.
- Decryption of the session key, followed by the decryption of the file payload.

2.3 Users and Characteristics

The system distinguishes between two primary roles during the workflow:

- **Receiver:** Initiates the setup by generating an RSA key pair. Exposes the Public Key while keeping the Private Key safely hidden in local memory. Ultimately decrypts the received payload to download the file.
- **Sender:** Retrieves the Receiver's Public Key, selects the confidential file, generates the AES session key, performs the hybrid encryption, and transfers the encrypted payload.

2.4 Operating Environment

Technology Stack:

- **HTML5 / CSS3:** Semantic web structure with a dark theme, neon purple accents (#b026ff), and glowing hover effects.
- **JavaScript (ES6+):** Handles async/await logic, ArrayBuffers, and UI state management.

- **WebCrypto API:** The core crypto engine for all native browser operations.

2.5 Design and Implementation Constraints

- **HTTPS Requirement:** The application must be served over HTTPS, as modern browsers strictly require a secure context to enable the WebCrypto API.
- **Client-Side Only:** No server-side scripting is permitted for handling keys or files to maintain the Zero-Knowledge Transport architecture.

2.6 Deployment

The application is a pure front-end application (HTML/CSS/JS), making it highly suitable for modern static hosting solutions like Vercel or GitHub Pages via auto-deploy processes.

3 Specific Requirements

3.1 External Interface Requirements

3.1.1 User Interfaces

The UI is designed with Neumorphic & Glow Aesthetics. It utilizes deep background colors contrasted with vibrant purple glows to guide user attention. The interface is logically split into left and right panels to visually mirror the Sender and Receiver.

- **Step-by-Step Guidance:** Visual cues numbered 1 through 6 to make complex cryptography easy to understand.
- **Real-time Feedback:** Dynamic badges such as "WAITING", "LOCKED", and "UNLOCKED" update according to the application's state.

3.1.2 Hardware & Software Interfaces

Operates on any standard computing device equipped with a modern web browser that fully supports HTML5 ArrayBuffers and the native WebCrypto API.

3.2 Functional Requirements (Detailed Workflow)

Phase 1: Setup (Receiver)

1. Generate Keys: The Receiver generates a 2048-bit RSA Public/Private key pair.
2. Share Public Key: The Receiver exposes their Public Key. The Private Key remains hidden.

Phase 2: Encryption (Sender)

3. Get Public Key: The Sender retrieves the Receiver's Public Key.
4. Select File: The Sender selects a confidential file.
5. Generate Session Key: The system generates a random 128-bit AES-CBC key and a 16-byte IV.
6. Encrypt File: The file is encrypted using the AES key.
7. Encrypt Session Key: The AES key is encrypted using the Receiver's RSA Public Key.
8. Transfer: Encrypted File, Encrypted AES Key, and IV are transferred to the Receiver.

Phase 3: Decryption (Receiver)

9. Decrypt Session Key: The Receiver uses their hidden RSA Private Key to decrypt the Encrypted AES Key.
10. Decrypt File: Using the decrypted AES key and IV, the file is decrypted.
11. Download: The Receiver downloads the restored confidential file.

3.3 Non-functional Requirements & Security Analysis

- **Zero-Knowledge Transport:** The network only sees the encrypted file and the encrypted AES key. Without the Receiver's Private Key, reading the file is mathematically infeasible.
- **OAEP Padding:** RSA is implemented with OAEP, protecting against chosen-ciphertext attacks.
- **Random IVs:** AES-CBC uses a randomly generated Initialization Vector for every file, preventing identical files from yielding identical ciphertexts.
- **Performance:** Achieving very fast speeds for large files compared to pure RSA setups.

4 System Design

4.1 End-to-End Operation Sequence

The system design relies heavily on a chronological execution of cryptographic functions. The data flow follows this sequence:

1. **[Receiver Dashboard]** Generates RSA Key Pair.
2. **[Sender]** → **[Receiver]** Requests Public Key.
3. **[Receiver]** → **[Sender]** Sends RSA Public Key.
4. **[Sender Dashboard]** Generates AES Session Key.
5. **[Sender Dashboard]** Encrypts File with AES Key.
6. **[Sender Dashboard]** Encrypts AES Key with RSA Public Key.
7. **[Sender]** → **[Receiver]** Transfers (Encrypted File + Encrypted AES Key + IV).
8. **[Receiver Dashboard]** Decrypts AES Key with RSA Private Key.
9. **[Receiver Dashboard]** Decrypts File with AES Key.

4.2 Algorithm Comparison Summary

Feature	RSA Only	AES Only	CryptDragon (Hybrid)
Performance on Large Files	Extremely Slow	Very Fast	Very Fast
Key Distribution Security	High	Low (Interception risk)	High
Complexity	Medium	Low	High
Best Use Case	Small data (Keys)	Local Disk Enc.	Secure File Transfer

5 Scope of Extension

While the current implementation successfully simulates the E2E encryption process within a single environment context, the system can be extended in the following ways:

- **Network Integration:** Integrating the core cryptographic engine with WebSockets or WebRTC to facilitate actual real-time file transfers between two separate physical devices.
- **Multi-File Support:** Extending the UI and processing logic to handle batch encryption of multiple files under a single session key or dynamic multiple keys.

- **Digital Signatures:** Adding a layer where the Sender signs the file with their own private key to ensure non-repudiation and verify sender authenticity.

Conclusion

The CryptDragon dashboard successfully demonstrates a modern, secure, and user-friendly approach to file encryption. It successfully resolves the traditional trade-offs between encryption speed and key distribution security, serving as an excellent model for secure web-based file-sharing applications.